

FlagYard Training Labs Writeup

Challenge Name: Mirror

Category: Web Exploitation

Difficulty: Medium

Mode: White Hat Pentesting (Code Files provided)

Code Files Given: [app.py](#) , templates (.html files) , main.css (style)

Static Analysis of [App.py](#) :

1. Sign up functionality

```
@app.route('/sign_up', methods=['GET', 'POST'])
def sign_up():
    if request.method == 'GET':
        return render_template('sign_up.html')

    username, password = request.form.get('username').strip(),
request.form.get('password')

    if username and password:
        try:
            db = db_connection()
            db.execute('insert into users values(lower(?), ?)',
(username, md5(password.encode()).hexdigest()))
            db.commit()
            flash(f'User {username} created', 'message')
            return redirect(url_for('login'))

        except sqlite3.IntegrityError:
            flash('Username is already registered', 'error')
        except sqlite3.Error:
            flash('An error occurred', 'error')
        return redirect(url_for('sign_up'))

    flash(f'Enter username and password', 'error')
    return redirect(url_for('sign_up'))
```

- > Accepts username and password
- > Strips username to remove trailing whitespaces
- > converts username to lowercase (for e.g Sagnik becomes sagnik)
- > Stores converted username and md5 hashed password in 'users' table

(**IntegrityError** : For pre-existing users in db)

2. Login Functionality:

```
@app.route('/')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'GET':
        return render_template('login.html')

    db = db_connection()
    username, password = request.form.get('username').strip(),
request.form.get('password')
    if username and password:
        logged_in = db.execute('select username from users where
username=lower(?) and password=?',
                                (username,
                                md5(password.encode()).hexdigest())).fetchone()
        if logged_in:
            session['user'] = logged_in['username'].capitalize()
            return redirect(url_for('account'))

        flash('Invalid credentials', 'error')
        return redirect(url_for('login'))

    flash('Enter username and password to login', 'error')
    return redirect(url_for('login'))
```

- > Accepts username and password (strips username for trailing whitespace)
- > Checks whether the username and password hash exists in DB.
- > If found then : establishes session key as the logged in capitalized username.

Note: `.capitalize()` function of python converts the initial letter of a word to a capital letter and rest of the letters stays lowercase (e.g sAgnik → Sagnik or sagnik → Sagnik)

> Once a session is established the user is routed to the account page.

3. Account route functionality

```
@app.route('/account')
def account():
    user = session.get('user')
    if user:
        if user == 'Flag':
            return render_template('account.html', user=user,
secret=open('flag.txt').read())
        return render_template('account.html', user=user)
    return redirect('/login')
```

> fetches session key checks if 'user' is Flag or not.

> If user == Flag : display flag

> Else : display the regular account page.

Vulnerability:

The login function is fetching username from the database, converting it , and establishing a session based on the converted username.

Nor the conversion (in login route) nor the sign up functionality converts the username to an uniform encoding.

Thus an attacker can take help of **variable encodings of the same letter to register the same username.**

Another form of **homoglyph attack**.

Exploitation Strategy:

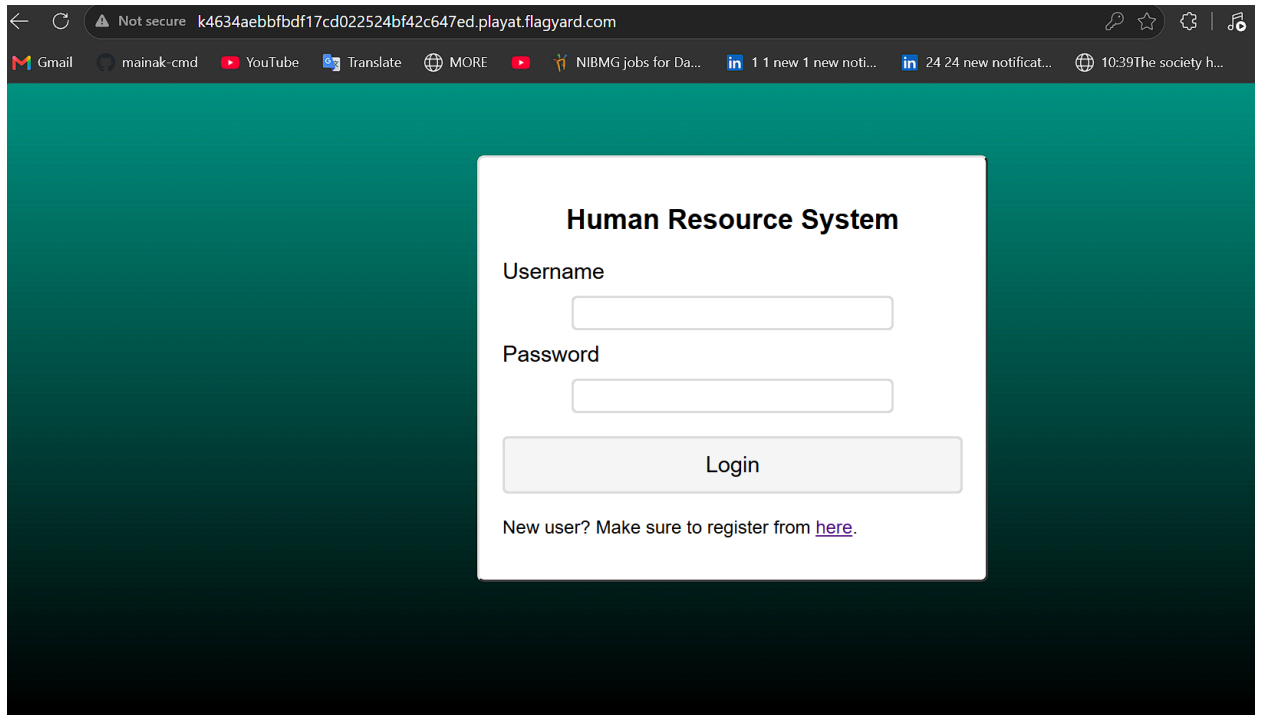
> register the flag user (use different encodings for only 'fl').

> Login as the registered spoofed user.

> Get the flag.

Dynamic Analysis

1. Establishing Baseline Regular Behaviour



A screenshot of a web browser displaying the login page of a 'Human Resource System'. The browser's address bar shows a URL starting with 'k4634aebbfdbf17cd022524bf42c647ed.playat.flagyard.com'. The page has a teal gradient background. A white login form is centered, containing the title 'Human Resource System', fields for 'Username' and 'Password', a 'Login' button, and a link for new users to register from 'here'.

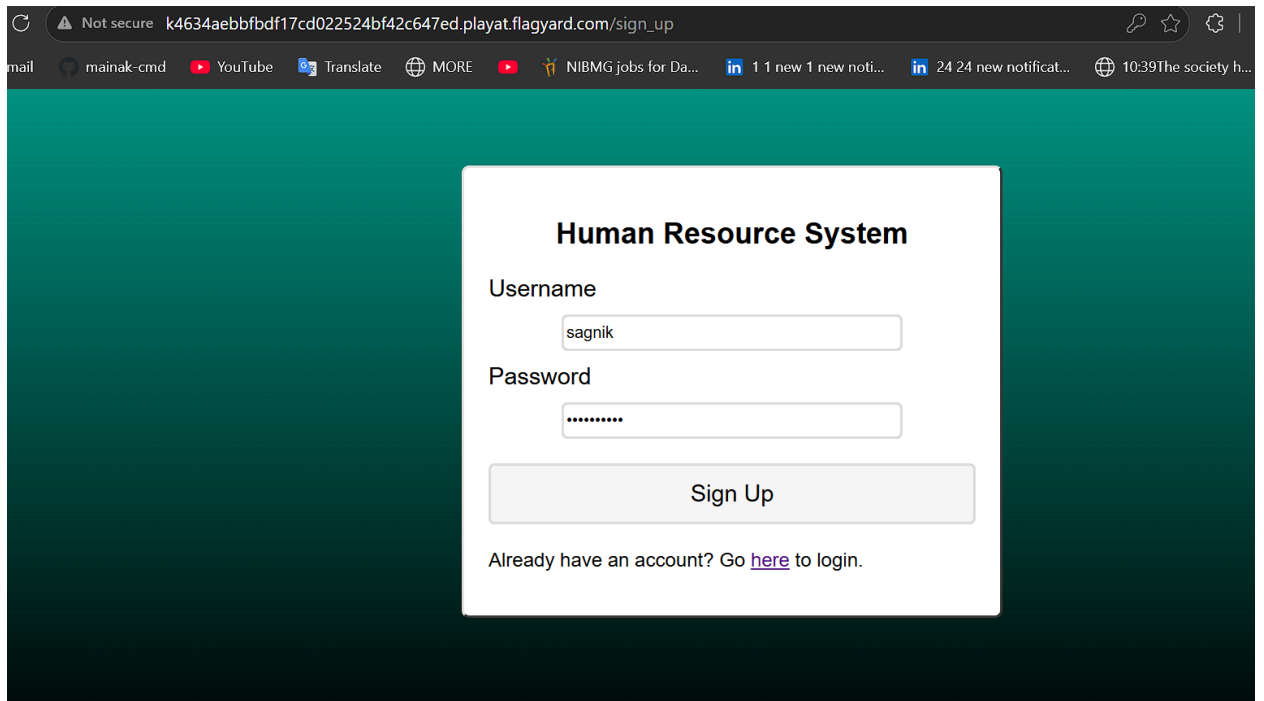
Human Resource System

Username

Password

Login

New user? Make sure to register from [here](#).



A screenshot of a web browser displaying the sign-up page of a 'Human Resource System'. The browser's address bar shows the URL 'k4634aebbfdbf17cd022524bf42c647ed.playat.flagyard.com/sign_up'. The page has a teal gradient background. A white sign-up form is centered, containing the title 'Human Resource System', fields for 'Username' (with 'sagnik' entered) and 'Password' (with masked characters), a 'Sign Up' button, and a link for existing users to login from 'here'.

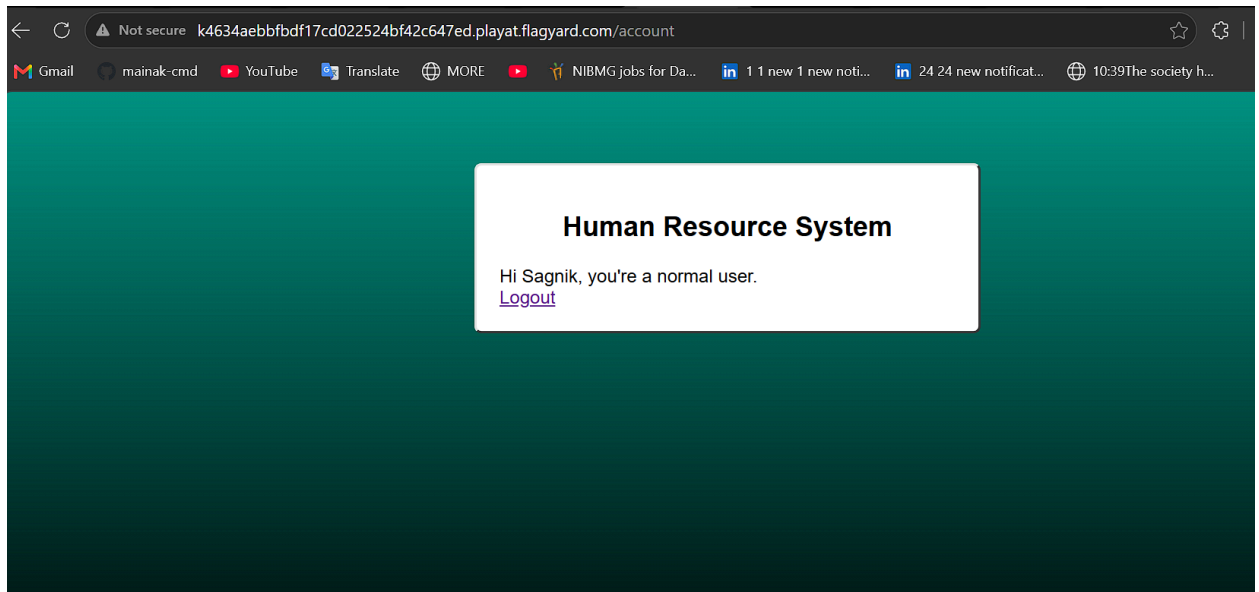
Human Resource System

Username

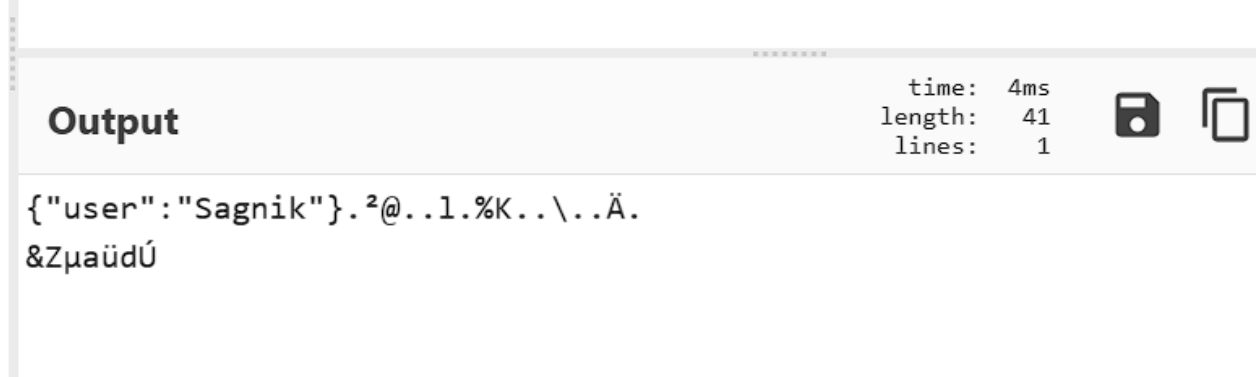
Password

Sign Up

Already have an account? Go [here](#) to login.



```
eyJ1c2VyIjoiU2Fnbm1rIn0.askCfg.WwMJUuRhFwSHc-SVDSZatWH8ZNo
```



Exploitation:

1. Spoofing Flag user

Using U+FB02 for fl

Reference : [“fl” U+FB02 Latin Small Ligature Fl Unicode Character](#)

Not secure k4634aebbfdbf17cd022524bf42c647ed.playat.flagyard.com/sign_up

Gmail mainak-cmd YouTube Translate MORE NIBMG jobs for Da... 1 1 new 1 new noti... 24 24 new notificat... 10:39The society h

Human Resource System

Username

Password

Sign Up

Already have an account? Go [here](#) to login.

2. Log in as the spoofed flag user.

Not secure k4634aebbfdbf17cd022524bf42c647ed.playat.flagyard.com/login

Gmail mainak-cmd YouTube Translate MORE NIBMG jobs for Da... 1 1 new 1 new noti... 24 24 new notificat... 10:39The society h

Human Resource System

Username

Password

Login

New user? Make sure to register from [here](#).

3. Flag displayed:

Human Resource System

Hi Flag, here is your secret:

FlagY{a465936087fffb0eac1029fc61064d48} .

[Logout](#)